

Customer Churn Platform

Beginners Masterclass Guide

Welcome! If you have zero background in IT, coding, or data science, you are in the exact right place. This guide is written by a master AI educator with 10 years of experience, designed to turn complex computer concepts into simple, everyday ideas.

By the end of this guide, you will understand exactly how every line of code in our Customer Churn Prediction Platform works, why it was written, and the science behind the decisions.

Chapter 1: The Absolute Basics of Programming (The Kitchen Analogy)

Writing code is exactly like writing a kitchen recipe.

- The Computer is a highly obedient cook who does exactly what you say, but has zero common sense. If you forget to tell the cook to turn on the stove, they will try to cook raw food.
- The Code is the set of step-by-step instructions.
- Variables are labelled storage containers. If you put sugar into a jar and write "sugar" on it, you can reference it later. In code, `monthly_charge = 70` means "create a box named `monthly_charge` and store the number 70 inside it."

The Core Parts of Python:

1. **Strings (Text):** Letters and words, wrapped in quotes, like `"Female"` or `"Month-to-month"`. Quotes tell Python, "This is text, don't try to run it as a command."
2. **Integers & Floats (Numbers):** Whole numbers like `12` (tenure in months) or decimals like `75.50` (monthly charges).
3. **Lists:** A shopping list of items wrapped in square brackets, like `["Gender", "Partner", "Contract"]`. It keeps items in a specific order.
4. **Dictionaries (Key-Value Pairs):** A real-world dictionary has a word and a definition. In Python, a dictionary has a "key" and a "value," wrapped in curly brackets, like `{"Gender": "Female", "Tenure": 12}`.
5. **Functions (`def`):** Think of a blender. You put in fruit (inputs), push button (run code), and get a smoothie (outputs). A function starts with `def` (definition), has a name, takes inputs (arguments), and uses `return` to give you the result.
6. **Imports (`import`):** Renting a specialized kitchen appliance instead of building it yourself. `import pandas` means "rent the database spreadsheet tool so we can use it."

Chapter 2: The Core Toolbox (What are these libraries?)

We use specialized tools (libraries) in our kitchen. Here is what they are in plain English:

- o Pandas (The Interactive Spreadsheet): Think of Pandas as Microsoft Excel inside Python. It lets us hold columns and rows of data, filter them, and merge different tables together.
 - o NumPy (The Calculator): A high-speed math engine. We use it to mark empty boxes (missing data) as "Not a Number" (`np.nan`).
 - o Scikit-Learn (The ML Factory): This is our tool for building predictive models. It contains the math code for scaling, data splitting, and models like Logistic Regression and Random Forest.
 - o XGBoost (The Supercharged Algorithm): A specialized, high-performance algorithm used by top data scientists to get maximum predictive accuracy.
 - o FastAPI (The Waiter): An API (Application Programming Interface) is like a waiter in a restaurant. When you order food (send customer data), the waiter takes it to the kitchen (our model), and brings back your meal (the prediction). FastAPI is a framework that makes this waiter incredibly fast.
 - o Streamlit (The Control Panel): Instead of command lines, Streamlit lets us build buttons, sliders, and forms in minutes using pure Python, giving business users a clean dashboard to click on.
 - o Joblib (The Freezer): When we train a model, it learns patterns. Joblib lets us "freeze" (serialize) that trained brain and save it to a file so we can reuse it instantly without retraining.
-

Chapter 3: What is Machine Learning? (The Flashcard Analogy)

Imagine you want to teach a child to identify a "high-risk churn customer."

You don't write rigid rules. Instead, you show the child 1,500 flashcards.

- o On the front of the card: customer details like tenure, charges, and contract type (these are called Features or clues).
- o On the back of the card: whether they cancelled their contract (this is called the Target or answer).

Training (Studying) vs. Testing (The Exam)

We split our 1,500 flashcards:

1. Training Set (80% / 1,200 cards): We let the model study these cards. It looks at the front (features) and back (churn label) to figure out patterns (e.g., "Oh, monthly contracts with high charges usually churn!").
2. Test Set (20% / 300 cards): The Exam. We cover the back of these 300 cards and ask the model to predict the answers. We then compare its predictions to the actual answers to see how smart the model is.

Meet the Models:

- o Logistic Regression (The Separating Line): Imagine drawing a line on a graph separating churners from non-churners. It calculates a single probability score. It's clean, fast, and simple.
- o Random Forest (The Council of Trees): Imagine asking 150 different experts (decision trees). Each expert looks at random clues (e.g., "Is tenure < 5?"). They all make a guess, and the majority vote wins. This handles complex, non-linear patterns.
- o XGBoost (The Study Group): Imagine a team study group. Expert 1 makes a guess. Expert 2 looks at where Expert 1 made mistakes and focuses *only* on correcting those mistakes. Expert 3 corrects Expert 2, and so on. It is incredibly accurate but can be complex.

Evaluation Metrics Made Simple:

- o Accuracy: What percentage of total exam questions did the model get right?

- o Precision: When the model yells "This customer is going to leave!", how often is it correct? (High precision means fewer false alarms).
- o Recall: Out of all the customers who actually left, how many did the model manage to catch? (High recall means we don't miss churners).
- o ROC-AUC: A score from 0 to 1 representing how good the model is at sorting customers by risk level. A 0.9 ROC-AUC means the model is excellent at ranking high-risk customers above low-risk ones.

Chapter 4: Line-by-Line Code Walkthrough (Every Token Explained)

Let's look at the actual code in the project, broken down line by line for absolute beginners.

File 1: `src/data\_ingestion.py` (Gathering the Ingredients)

```
import os
import pandas as pd
import sqlite3
import logging
```

- o Why this is written: We start by importing our tools. `os` helps us talk to the computer's folders. `pd` (Pandas) is our spreadsheet tool. `sqlite3` connects to our database. `logging` prints log files so we know what's happening.

```
def ingest_data(csv_path=None, db_path=None):
```

- o Why this is written: We define a function (blender) named `ingest\_data` that takes two folders as input parameters.

```
    df_demographics = pd.read_csv(csv_path)
```

- o Why this is written: Calls Pandas (`pd`) to read the CSV table (`read\_csv`) and saves the result in a variable named `df\_demographics`.

```
    conn = sqlite3.connect(db_path)
    df_services = pd.read_sql_query("SELECT * FROM services", conn)
    conn.close()
```

- o Why this is written: Opens a database pipeline (`connect`), runs a SQL query to select all records (`SELECT \*`) from the `services` table, stores it as `df\_services`, and closes the connection to save memory.

```
    merged_df = pd.merge(df_demographics, df_services, on="CustomerId", how="inner")
    return merged_df
```

- o Why this is written: Merges the two tables together using `CustomerId` as the matching key. It returns the combined table as the final smoothie.

File 2: `src/preprocessing.py` (Preparing the Ingredients)

Computer models are like picky eaters—they only eat numbers between -3 and 3, and they don't understand words like "Male" or "Yes". This file translates everything into numbers.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```
from sklearn.pipeline import Pipeline
```

- Why this is written: Imports from Scikit-Learn. `SimpleImputer` fills empty cells. `StandardScaler` rescales monthly charges (e.g. \$70 becomes 0.5). `OneHotEncoder` turns categories like `Contract` into binary columns (Month-to-month = 1, Two-year = 0). `Pipeline` chains them together.

```
def clean_data(df):
    df = df.copy()
    if "TotalCharges" in df.columns:
        df["TotalCharges"] = df["TotalCharges"].replace(r'^\s*$', np.nan, regex=True)
        df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
        df["TotalCharges"] = df["TotalCharges"].fillna(df["MonthlyCharges"] * df["Tenure"])
```

- Why this is written: We clean the `TotalCharges` column. If a cell is blank (`r'^\s\*\$'`), we turn it into an empty cell (`np.nan`), convert the column to numbers (`to\_numeric`), and calculate the missing values by multiplying monthly charges by tenure.

```
    if "Churn" in df.columns:
        df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0}).fillna(0).astype(int)
    return df
```

- Why this is written: We convert our target labels from text ("Yes"/"No") into numbers (1/0) so the math models can learn from them.

```
num_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

- Why this is written: Sets up a numerical recipe: if a number is missing, fill it with the middle number (median), then scale all values so they are similarly sized.

```
cat_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])
```

- Why this is written: Sets up a text recipe: fill missing words with the most common word, then expand categories into 1/0 columns (OneHotEncoding).

File 3: `src/train.py` (Teaching the Brain)

```
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=42, stratify=y
)
```

- Why this is written: Splits our features (`X\_processed`) and answers (`y`) into an 80/20 train/test split. `random\_state=42` is a seed that ensures the split is identical every time we run the code. `stratify=y` guarantees that the churn rate is identical in both splits.

```
models = {
    "LogisticRegression": LogisticRegression(max_iter=1000, random_state=42),
    "RandomForest": RandomForestClassifier(n_estimators=150, max_depth=8, random_state=42),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric="logloss", random_state=42)
}
```

- Why this is written: Defines a dictionary holding the three model objects we want to train and compare.

```
for name, model in models.items():
    model.fit(X_train, y_train)
```

- o Why this is written: Loops through each model and runs `.fit()`, which runs the optimization mathematics to train the model on our training data.

```
joblib.dump(best_model_obj, best_model_path)
```

- o Why this is written: Saves the highest-scoring model to a physical file so our API can load it later.

File 4: ``src/api.py`` (The Restaurant Waiter)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    startup_event()
    yield
```

- o Why this is written: The lifespan context manager runs `startup_event()` once when the API starts. This loads the saved model (`best_model.joblib``) into memory immediately, rather than loading it on every user request, making the system fast.

```
class CustomerPayload(BaseModel):
    Gender: str
    Tenure: int
    MonthlyCharges: float
    # ...
```

- o Why this is written: A Pydantic schema class. It defines the exact input structure clients must send. FastAPI automatically checks incoming requests against this schema and rejects malformed inputs.

```
@app.post("/predict", response_model=BatchPredictionResponse)
def predict(payload: List[CustomerPayload]):
```

- o Why this is written: Sets up a POST endpoint ``/predict`` which accepts a list of customer data payloads.

```
    records = [item.model_dump() for item in payload]
    df_input = pd.DataFrame(records)
    df_clean = clean_data(df_input)
    X_processed = preprocessor.transform(df_clean)
    probs = best_model.predict_proba(X_processed)[: , 1]
```

- o Why this is written: When a request arrives, we convert the payloads to a Pandas DataFrame, clean it, transform it using our saved preprocessor, and run the model's prediction probability. We format the predictions and return them as JSON.

File 5: ``src/dashboard.py`` (The Interactive Dashboard)

```
import streamlit as st
import requests
import joblib
```

- o Why this is written: Streamlit (``st``) creates the UI, ``requests`` calls the FastAPI backend, and ``joblib`` loads files locally.

```
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

- o Why this is written: Tells Python to look in the workspace root directory so it can import our preprocessing functions without generating a `ModuleNotFoundError`.

```
def predict_churn_local(payload):
```

- o Why this is written: A fallback prediction function. If the FastAPI backend is offline, this function loads `best\_model.joblib` and `preprocessor.joblib` locally from the folders to calculate the churn probability directly.

```
page = st.sidebar.radio("Go To", ["Overview & Insights", "Single Customer Predictor", "Model Diagnostic H
```

- o Why this is written: Creates a sidebar radio list of pages. Streamlit automatically switches the screen display based on the selected value.

```
    with st.form("churn_prediction_form"):
        # inputs
        submit_btn = st.form_submit_button("Compute Churn Probability")
```

- o Why this is written: Creates a visual input form. When the user clicks the submit button, it compiles all slider and select box selections into a payload dictionary and calls the predictor to show the churn percentage.

```
---
```

Chapter 5: Deployment & Containerization (Docker)

When deploying applications, setting up Python, packages, and system files manually on a server is slow and prone to errors.

The Shipping Container Analogy

Before standard shipping containers, cargo had to be loaded piece-by-piece, which was highly inefficient. Shipping containers standardise the format.

Docker does this for software. It packs our code, the Python version, and our library dependencies into an isolated Docker Image (the container). This image runs identically on your computer, a Windows server, or a cloud server like AWS.

- o Dockerfile: A recipe text file that tells Docker how to build the container (e.g. start with Python 3.11, copy requirements.txt, run pip install, copy code files, and set startup commands).

- o Docker Compose (`docker-compose.yml`): Coordinates multiple containers. In our project, we have two containers running side-by-side: `api` (FastAPI) and `dashboard` (Streamlit). Docker Compose spins them up together, links their network so they can talk, and maps local folders (`volumes`) so they share datasets and models in real time.